CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

CASE STUDY REPORT
ON
COLLEGE MANAGEMENT SYSTEM

(Using C++ Object-Oriented Programming)

Program Name: BCA(Hons)(Data Science)

Subject Code: 25CAH-201

C++ OOP Mini Project Case Study

Submitted by:

Name: TARANJOT SINGH

UID: 25BCD10103

Submitted to:

Name: Mr. Jitendra Kumar

Designation: Assistant Professor

PROJECT OVERVIEW

This mini-project report presents a working, menu-driven College Management System developed in C++17 using the principles of Object-Oriented Programming (OOP). It is designed as a console application for managing student records, faculty records, academic courses, student enrolments, and grades.

The structure of this report follows the submitted Chandigarh University sample: Introduction, Objectives / Problem Statement, Modules and Tools, Code with Outputs, and Conclusion. The cover retains the University Institute of Computing style and branding from the supplied front page.

Report contents

Section	What it contains
1. Introduction	Purpose, scope and real-world academic context
2. Objectives / Problem Statement	Problem, functional and quality objectives
3. Modules, Tools and System Design	Seven modules, technology stack, class structure, storage
4. Code with Outputs	Compilation steps, complete C++ source, verified output
5. Testing and Learning Outcomes	Test scenarios, OOP coverage, key results
6. Conclusion	Achievements, limitations and improvements

Project at a glance

Item	Project specification
Application type	Menu-driven command-line application
Language	ISO C++17
OOP mechanisms	Abstraction, encapsulation, inheritance, polymorphism
Stored records	Students, faculty, courses and enrollments
Persistence	Four local text files; records restored at startup
Demonstration	Two sample students, two faculty, two courses, two enrolments

Submission information: Prepared by TARANJOT SINGH (UID 25BCD10103), subject code 25CAH-201; submitted to Mr. Jitendra Kumar, Assistant Professor.

1. Introduction

College offices routinely manage academic information such as who is registered, which teacher is assigned to a course, and which subjects each student is studying. When these records are scattered across paper forms or separate spreadsheets, updates become repetitive and it is easy to introduce duplicate identifiers or inconsistent records.

The proposed College Management System centralizes these operations within one C++ application. An operator can add student and faculty details, create courses, enrol students in courses, assign letter grades, search by student ID, and view summary reports. Data is stored locally so that records remain available when the program is closed and reopened.

The project intentionally focuses on a small, understandable academic management workflow rather than attempting to replace a complete enterprise university ERP platform. It demonstrates important OOP practices using connected real-world entities: Person, Student, Faculty, Course, Enrollment, and CollegeManagement.

1.1 Real-world example

During semester registration, a BCA student with student ID 103 is admitted to the course 25CAH-201 - Object Oriented Programming. The course has an assigned faculty member with ID 501. The operator first adds the faculty record, then creates the course, then adds the student and enrols them. After evaluation, grade A is entered and stored. Restarting the program displays the same records.

1.2 Scope and boundaries

- Maintain and display student information including ID, name, program and semester.
- Maintain faculty details including department and designation.
- Assign existing faculty to courses and define the course credit count.
- Enrol a student in an existing course and record an A/B/C/D/F grade.
- Perform record lookup, summaries and automatic file-based saving.
- Exclude attendance, fee collection, timetable planning, role-based login and online synchronization from this version.

2. Objectives / Problem Statement

2.1 Problem statement

Manual student and faculty registers are difficult to maintain when the number of records grows. Repeating information across multiple sheets increases the risk of duplication, missed enrolments and inconsistent course assignments. A simple program is needed to maintain academic records in one place and verify relationships before updates are accepted.

2.2 Project objectives

- To design an easy-to-use menu-driven application for college academic administration.
- To represent related entities with reusable classes and clear data members.
- To demonstrate inheritance and runtime polymorphism through Student and Faculty classes derived from Person.

- To prevent duplicate student IDs, duplicate faculty IDs, duplicate course codes and repeated enrolments.
- To validate course-to-faculty and enrolment-to-student/course relationships.
- To save and reload records using C++ file streams and quoted text strings.
- To display meaningful lists, student search results, grade records and a summary report.

2.3 Expected benefits

The system reduces repeated manual entry, provides reliable identifiers, protects a few key relationships from invalid entries and gives a quick summary of the data. Its small scope also makes it appropriate for a classroom lab and viva demonstration.

3. Modules, Tools and System Design

3.1 Modules

Module	Function
1. Student Management	Create and display student ID, full name, program and semester; search by ID.
2. Faculty Management	Create and display ID, name, department and designation.
3. Course Management	Add course code, title, credits and a validated faculty ID.
4. Enrollment Management	Link existing students with existing courses; avoid repeated enrolment.
5. Grade Management	Update the existing enrolment with a valid letter grade.
6. Reporting	Display all records and counts of students, faculty, courses and enrollments.
7. File Management	Load records on startup and rewrite the local data files after a change.

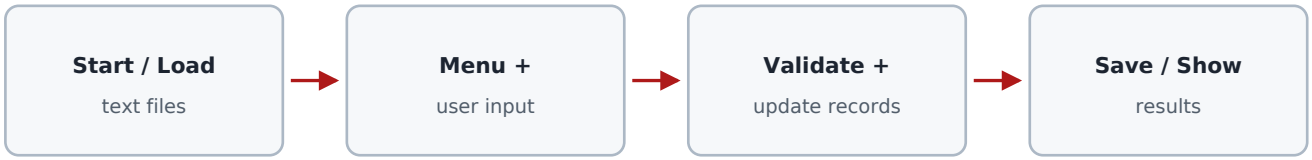
3.2 Tools and technologies used

Tool / Technology	Purpose
C++17	Implementation of application logic and OOP classes.
g++ / MinGW-w64	Compile code with C++17 language support.
Visual Studio Code or text editor	Edit the .cpp source file.
Windows PowerShell / Terminal	Compile, run and interact with the console menu.
C++ standard library	vector, fstream, iomanip, string and input handling.
Text files	Store data persistently without an external database.

3.3 OOP concepts demonstrated

Concept	Where used in the project
Class and objects	Student, Faculty, Course and CollegeManagement classes.
Encapsulation	Record fields are private/protected; access occurs through public methods.
Inheritance	Student and Faculty publicly inherit abstract base class Person.
Abstraction	Person declares a pure virtual display() function.
Runtime polymorphism	display() is called through a Person reference and resolves to the derived implementation.
Constructors / destructor	Derived records are initialized through constructors; Person has a virtual destructor.
Composition / aggregation	CollegeManagement maintains vectors of related record objects.

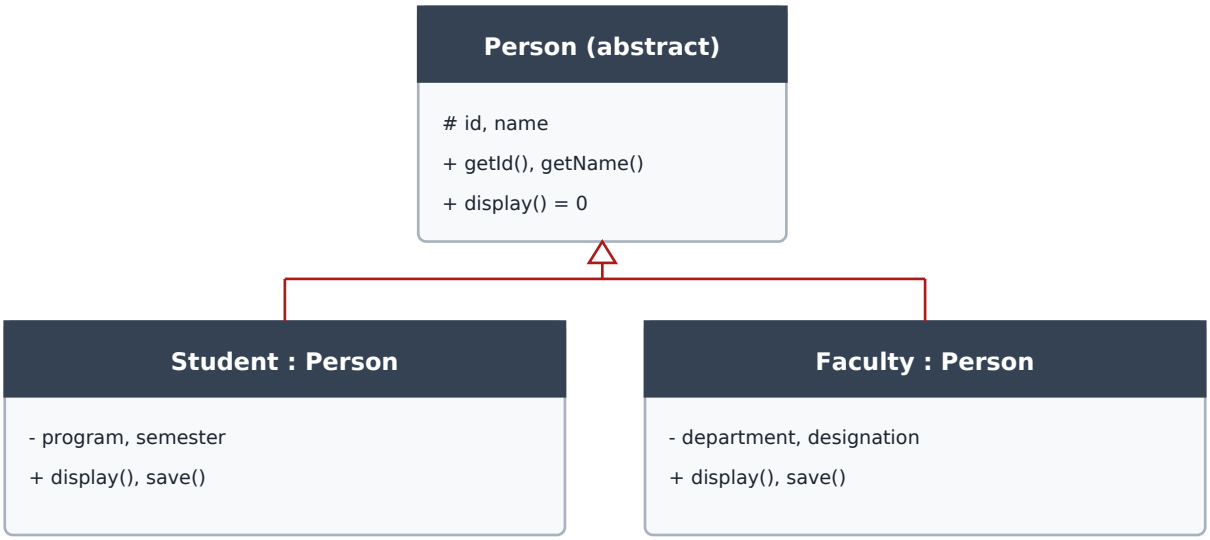
3.4 Application workflow



File persistence: students.txt | faculty.txt | courses.txt | enrollments.txt

At startup, the program reads saved records from four files. It then displays twelve menu choices. Each modification first checks record identifiers and relationships, updates the corresponding vector, and saves the dataset. On request, the program displays details or statistical counts.

3.5 Class structure (simplified UML-style view)



The abstract Person class serves as a common interface for Student and Faculty. The CollegeManagement class coordinates records and menu operations. Course and Enrollment represent course details and student-to-course registration.

3.6 Persistent file design

File	Stored fields
students.txt	ID, quoted student name, quoted program, semester
faculty.txt	ID, quoted faculty name, quoted department, quoted designation
courses.txt	Quoted course code, quoted course title, credits, faculty ID
enrollments.txt	Student ID, quoted course code, quoted grade

Note: Fields with spaces are written using `std::quoted`, which allows the same values to be read back accurately. IDs in the running example are short numeric academic identifiers; the author UID on the cover is a separate university identifier.

4. Code with Outputs

4.1 Compilation and execution

Save the supplied companion source as `College_Management_System.cpp`. Compile using a C++17 compiler and run the resulting application. Data files are created in the current working directory after the first successful change.

```
g++ -std=c++17 -Wall -Wextra College_Management_System.cpp -o college_manager
./college_manager              # Linux / macOS
college_manager.exe            # Windows PowerShell / CMD
```

The application was compiled with `g++ -std=c++17 -Wall -Wextra -pedantic` and its sample workflow was executed. The next subsection reproduces the complete, tested source code.

4.2 Complete source code (C++17)

```
LINE   SOURCE CODE
001  // COLLEGE MANAGEMENT SYSTEM - C++ OOP MINI PROJECT
002  // Prepared by: TARANJOT SINGH | UID: 25BCD10103
003  // Course code: 25CAH-201
004  // Compile: g++ -std=c++17 College_Management_System.cpp -o college_manager
005  // Run: ./college_manager  (Windows: college_manager.exe)
006
007  #include <iostream>
008  #include <fstream>
009  #include <iomanip>
010  #include <limits>
011  #include <string>
012  #include <vector>
013  using namespace std;
014
015  // Base class: abstraction and runtime polymorphism.
016  class Person {
017  protected:
018      int id;
019      string name;
020  public:
021      Person(int number, string fullName) : id(number), name(fullName) {}
022      virtual ~Person() = default;
023      int getId() const { return id; }
024      string getName() const { return name; }
025      virtual void display() const = 0;
026  };
027
028  // Derived class: inheritance from Person.
029  class Student : public Person {
030      string program;
031      int semester;
032  public:
033      Student(int number, string fullName, string course, int sem)
034          : Person(number, fullName), program(course), semester(sem) {}
035      void display() const override {
036          cout << "ID: " << id << " | Name: " << name
037               << " | Program: " << program
038               << " | Semester: " << semester << '\n';
039      }
040      void save(ostream& out) const {
041          out << id << ' ' << quoted(name) << ' ' << quoted(program)
042              << ' ' << semester << '\n';
043      }
044  };
045
046  // Derived class: inheritance and method overriding.
047  class Faculty : public Person {
048      string department;
049      string designation;
050  public:
051      Faculty(int number, string fullName, string dept, string job)
052          : Person(number, fullName), department(dept), designation(job) {}
053      void display() const override {
054          cout << "ID: " << id << " | Name: " << name
055               << " | Department: " << department
056               << " | Designation: " << designation << '\n';
057      }
058      void save(ostream& out) const {
059          out << id << ' ' << quoted(name) << ' '
```

```
LINE   SOURCE CODE
060             << quoted(department) << ' ' << quoted(designation) << '\n';
061     }
062 };
063
064 class Course {
065     string code;
066     string title;
067     int credits;
068     int facultyId;
069 public:
070     Course(string courseCode, string courseTitle, int creditCount, int teacherId)
071         : code(courseCode), title(courseTitle), credits(creditCount), facultyId(teacherId) {}
072     string getCode() const { return code; }
073     void display() const {
074         cout << "Code: " << code << " | Title: " << title
075              << " | Credits: " << credits
076              << " | Faculty ID: " << facultyId << '\n';
077     }
078     void save(ostream& out) const {
079         out << quoted(code) << ' ' << quoted(title)
080             << ' ' << credits << ' ' << facultyId << '\n';
081     }
082 };
083
084 struct Enrollment {
085     int studentId;
086     string courseCode;
087     string grade; // NA until a grade is recorded.
088 };
089
090 class CollegeManagement {
091     vector<Student> students;
092     vector<Faculty> faculty;
093     vector<Course> courses;
094     vector<Enrollment> enrollments;
095
096     static int readInt(const string& prompt) {
097         int value;
098         while (true) {
099             cout << prompt;
100             if (cin >> value) return value;
101             if (cin.eof()) return -1;
102             cout << "Invalid number. Please try again.\n";
103             cin.clear();
104             cin.ignore(numeric_limits<streamsize>::max(), '\n');
105         }
106     }
107     static string readText(const string& prompt) {
108         cout << prompt;
109         string answer;
110         getline(cin >> ws, answer);
111         return answer;
112     }
113     Student* findStudent(int number) {
114         for (auto& item : students)
115             if (item.getId() == number) return &item;
116         return nullptr;
117     }
118     Faculty* findFaculty(int number) {
119         for (auto& item : faculty)
120             if (item.getId() == number) return &item;
```

```
LINE   SOURCE CODE
121          return nullptr;
122      }
123      Course* findCourse(const string& code) {
124          for (auto& item : courses)
125              if (item.getCode() == code) return &item;
126          return nullptr;
127      }
128      void load() {
129          int id, sem, credits, teacherId;
130          string name, program, dept, designation, code, title, grade;
131          ifstream s("students.txt");
132          while (s >> id >> quoted(name) >> quoted(program) >> sem)
133              students.emplace_back(id, name, program, sem);
134          ifstream f("faculty.txt");
135          while (f >> id >> quoted(name) >> quoted(dept) >> quoted(designation))
136              faculty.emplace_back(id, name, dept, designation);
137          ifstream c("courses.txt");
138          while (c >> quoted(code) >> quoted(title) >> credits >> teacherId)
139              courses.emplace_back(code, title, credits, teacherId);
140          ifstream e("enrollments.txt");
141          while (e >> id >> quoted(code) >> quoted(grade))
142              enrollments.push_back({id, code, grade});
143      }
144      void save() const {
145          ofstream s("students.txt"), f("faculty.txt");
146          ofstream c("courses.txt"), e("enrollments.txt");
147          if (!s || !f || !c || !e) {
148              cerr << "Error: Could not write data files.\n";
149              return;
150          }
151          for (const auto& item : students) item.save(s);
152          for (const auto& item : faculty) item.save(f);
153          for (const auto& item : courses) item.save(c);
154          for (const auto& item : enrollments)
155              e << item.studentId << ' ' << quoted(item.courseCode)
156                << ' ' << quoted(item.grade) << '\n';
157      }
158      void addStudent() {
159          int id = readInt("Enter student ID: ");
160          if (id <= 0 || findStudent(id)) {
161              cout << "Invalid or duplicate student ID.\n"; return;
162          }
163          string name = readText("Enter name: ");
164          string program = readText("Enter program: ");
165          int semester = readInt("Enter semester (1-8): ");
166          if (semester < 1 || semester > 8) {
167              cout << "Semester must be between 1 and 8.\n"; return;
168          }
169          students.emplace_back(id, name, program, semester);
170          save();
171          cout << "Student added successfully.\n";
172      }
173      void addFaculty() {
174          int id = readInt("Enter faculty ID: ");
175          if (id <= 0 || findFaculty(id)) {
176              cout << "Invalid or duplicate faculty ID.\n"; return;
177          }
178          string name = readText("Enter faculty name: ");
179          string dept = readText("Enter department: ");
180          string job = readText("Enter designation: ");
181          faculty.emplace_back(id, name, dept, job);
```

```
LINE   SOURCE CODE
182          save();
183          cout << "Faculty added successfully.\n";
184      }
185      void addCourse() {
186          string code = readText("Enter course code: ");
187          if (findCourse(code)) {
188              cout << "Course code already exists.\n"; return;
189          }
190          string title = readText("Enter course title: ");
191          int credits = readInt("Enter credits (1-6): ");
192          int facultyId = readInt("Enter assigned faculty ID: ");
193          if (credits < 1 || credits > 6 || !findFaculty(facultyId)) {
194              cout << "Invalid credits or faculty ID.\n"; return;
195          }
196          courses.emplace_back(code, title, credits, facultyId);
197          save();
198          cout << "Course added successfully.\n";
199      }
200      void enrollStudent() {
201          int id = readInt("Enter student ID: ");
202          string code = readText("Enter course code: ");
203          if (!findStudent(id) || !findCourse(code)) {
204              cout << "Student or course not found.\n"; return;
205          }
206          for (const auto& item : enrollments)
207              if (item.studentId == id && item.courseCode == code) {
208                  cout << "Student is already enrolled.\n"; return;
209              }
210          enrollments.push_back({id, code, "NA"});
211          save();
212          cout << "Enrollment successful.\n";
213      }
214      void recordGrade() {
215          int id = readInt("Enter student ID: ");
216          string code = readText("Enter course code: ");
217          for (auto& item : enrollments) {
218              if (item.studentId == id && item.courseCode == code) {
219                  string grade = readText("Enter grade (A/B/C/D/F): ");
220                  if (grade != "A" && grade != "B" && grade != "C"
221                      && grade != "D" && grade != "F") {
222                      cout << "Invalid grade.\n"; return;
223                  }
224                  item.grade = grade;
225                  save();
226                  cout << "Grade saved successfully.\n"; return;
227              }
228          }
229          cout << "Enrollment not found.\n";
230      }
231      void viewStudents() const {
232          cout << "\n--- STUDENT RECORDS ---\n";
233          if (students.empty()) cout << "No records found.\n";
234          // Calls overridden display(): runtime polymorphism.
235          for (const auto& student : students) {
236              const Person& person = student;
237              person.display();
238          }
239      }
240      void viewFaculty() const {
241          cout << "\n--- FACULTY RECORDS ---\n";
242          if (faculty.empty()) cout << "No records found.\n";
```

```
LINE   SOURCE CODE
243          for (const auto& teacher : faculty) {
244              const Person& person = teacher;
245              person.display();
246          }
247      }
248      void viewCourses() const {
249          cout << "\n--- COURSE RECORDS ---\n";
250          if (courses.empty()) cout << "No records found.\n";
251          for (const auto& course : courses) course.display();
252      }
253      void viewEnrollments() const {
254          cout << "\n--- ENROLLMENT RECORDS ---\n";
255          if (enrollments.empty()) cout << "No records found.\n";
256          for (const auto& entry : enrollments)
257              cout << "Student ID: " << entry.studentId
258                   << " | Course: " << entry.courseCode
259                   << " | Grade: " << entry.grade << '\n';
260      }
261      void searchStudent() {
262          int id = readInt("Enter student ID to search: ");
263          Student* student = findStudent(id);
264          if (student) student->display();
265          else cout << "Student not found.\n";
266      }
267      void summary() const {
268          cout << "\n--- COLLEGE SUMMARY ---\n"
269               << "Total students: " << students.size() << '\n'
270               << "Total faculty: " << faculty.size() << '\n'
271               << "Total courses: " << courses.size() << '\n'
272               << "Total enrollments: " << enrollments.size() << '\n';
273      }
274  public:
275      CollegeManagement() { load(); }
276      void run() {
277          int choice;
278          do {
279              cout << "\n========== COLLEGE MANAGEMENT SYSTEM ==========\n"
280                   << "1. Add Student            2. Add Faculty\n"
281                   << "3. Add Course             4. Enroll Student\n"
282                   << "5. Record Grade           6. View Students\n"
283                   << "7. View Faculty           8. View Courses\n"
284                   << "9. View Enrollments      10. Search Student\n"
285                   << "11. Summary Report       12. Exit\n";
286              choice = readInt("Enter your choice: ");
287              switch (choice) {
288                  case 1: addStudent(); break;
289                  case 2: addFaculty(); break;
290                  case 3: addCourse(); break;
291                  case 4: enrollStudent(); break;
292                  case 5: recordGrade(); break;
293                  case 6: viewStudents(); break;
294                  case 7: viewFaculty(); break;
295                  case 8: viewCourses(); break;
296                  case 9: viewEnrollments(); break;
297                  case 10: searchStudent(); break;
298                  case 11: summary(); break;
299                  case 12: cout << "Data saved. Goodbye!\n"; break;
300                  default: cout << "Invalid choice. Try again.\n";
301              }
302          } while (choice != 12 && choice != -1);
303      }
```

```
LINE   SOURCE CODE
304  };
305
306  int main() {
307      CollegeManagement college;
308      college.run();
309      return 0;
310  }
```

4.3 Program output (from verified execution)

The following terminal-style panels reproduce relevant lines from an actual test session of the compiled program. They are formatted from captured console text for readability; they are not photographs of a physical terminal window.

Output 1. Menu of the compiled C++ application

C++ console / verified program output

```
========== COLLEGE MANAGEMENT SYSTEM ==========
1. Add Student            2. Add Faculty
3. Add Course             4. Enroll Student
5. Record Grade           6. View Students
7. View Faculty           8. View Courses
9. View Enrollments      10. Search Student
11. Summary Report       12. Exit
```

Figure 1: Captured console text from the successful C++17 execution.

Output 2. Student and faculty records

C++ console / verified program output

```
--- STUDENT RECORDS ---
ID: 103 | Name: Taranjot Singh | Program: BCA Data Science | Semester: 3
ID: 104 | Name: Simran Kaur | Program: BCA Data Science | Semester: 3

--- FACULTY RECORDS ---
ID: 501 | Name: Jitendra Kumar | Department: Computer Applications | Designation: Assistant Professor
ID: 502 | Name: Shilpa Mahajan | Department: Data Science | Designation: Assistant Professor
```

Figure 2: Captured console text from the successful C++17 execution.

Output 3. Courses and enrolments

C++ console / verified program output

```
--- COURSE RECORDS ---
Code: 25CAH-201 | Title: Object Oriented Programming | Credits: 4 | Faculty ID: 501
Code: 25CAP-204 | Title: Database Management System | Credits: 4 | Faculty ID: 502

--- ENROLLMENT RECORDS ---
Student ID: 103 | Course: 25CAH-201 | Grade: A
Student ID: 104 | Course: 25CAP-204 | Grade: NA
```

Figure 3: Captured console text from the successful C++17 execution.

Output 4. Saved summary report

C++ console / verified program output

```
--- COLLEGE SUMMARY ---
Total students: 2
Total faculty: 2
Total courses: 2
Total enrollments: 2
```

Figure 4: Captured console text from the successful C++17 execution.

Output 5. Verified data loaded after reopening the program

C++ console / verified program output

```
--- COLLEGE SUMMARY ---
Total students: 2
Total faculty: 2
Total courses: 2
Total enrollments: 2
```

Figure 5: Captured console text from the successful C++17 execution.

5. Testing and Learning Outcomes

5.1 Test cases and results

Test case	Action and expected result	Observed status
TC-01	Add faculty ID 501; faculty saved.	PASS

Test case	Action and expected result	Observed status
TC-02	Add student ID 103; student saved.	PASS
TC-03	Add course 25CAH-201 for faculty 501; course saved.	PASS
TC-04	Enrol student 103 into 25CAH-201; enrolment saved.	PASS
TC-05	Record grade A for existing enrolment.	PASS
TC-06	List students, faculty, courses and enrollments.	PASS
TC-07	Search for student ID 103.	PASS
TC-08	Try to add duplicate student ID 103; show rejection.	PASS
TC-09	Reopen the application; previously saved counts remain.	PASS
TC-10	Summary with 2 students, 2 faculty, 2 courses and 2 enrollments.	PASS

5.2 Main learning outcomes

- Implemented an academic information system using C++ classes, objects and reusable member functions.
- Demonstrated abstraction, inheritance, encapsulation and virtual-function polymorphism in a practical project.
- Used vectors, C++ file streams, input validation, searching and persistent record storage to manage related entities.

5.3 Limitations and future enhancements

The current version stores records in plain local text files, so it is intended for a single operator and a small academic dataset. It does not include passwords, concurrent users, automatic backups, fee transactions or a graphical interface. Possible future enhancements include SQLite/MySQL integration, user authentication, attendance and fee modules, timetable management, and a GUI built using Qt.

6. Conclusion

The College Management System demonstrates how C++ and Object-Oriented Programming can solve a common academic record-management problem. It organizes student, faculty, course and enrollment information through related classes and provides user-friendly menu commands for record creation, display, grade recording, student search and summary reports.

The program applies important OOP ideas in actual code: the abstract Person base class provides a common interface, Student and Faculty inherit from Person, overridden display() methods demonstrate runtime polymorphism, and private data members protect the internal representation. C++ file handling preserves records across application restarts.

Sample tests confirmed successful compilation, record insertion, search, grade updates, duplicate-ID rejection and persistence. As a mini-project, the system provides a practical foundation for learning C++ OOP and can be extended into a more advanced college administration application.

References and supporting material

1. ISO C++17 standard library features: iostream, fstream, iomanip, vector and string.
2. Course-based Object-Oriented Programming concepts: classes, inheritance, abstraction and polymorphism.
3. Companion file included with this submission: `College_Management_System.cpp`.

Prepared by: TARANJOT SINGH | UID: 25BCD10103 | Subject Code: 25CAH-201